Name	Ali Imran
Subject	DevOps Capstone Project

Index

Index **2**
Project Overview 4
Create a GitHub Repository 5
Create a User Story Template 6
Assemble Your Product Backlog 7
Triage New Issues 8
Refine Your Product Backlog 9
#1 9
#2 9
#3 9
#4 10
#5 10
Build First Sprint from Product Backlog 12
Initialize Development Environment **13**
Project Overview **15**
REST API **16**
Implement First User Story **17**
Create a REST API with Flask **19**
Run the REST service **22**
Sprint Review **23**
Demo Create Account 23
Demo List All Accounts 23
Demo Read an Account 23
Demo Update an Account 23
Demo Delete an Account 24
Planning **25**
Story 1: Need the Ability to Automate Continuous Integration Checks 25
Story 2: Need to Add Security Headers and CORS Policies 26
Initialize Development Environment **27**
Pick Up the First Story **29**
Create a Workflow **30**
Create a Job **31**
Define Required Services **32**
Check Out Code and Install Dependencies **35**
Add Linting **37**
Add Unit Testing **39**
Create a Badge **42**
Make a Pull Request **43**

Pick Up the Next Story **44**
Observe the Current Behavior **45**
Write a Test Case **46**
Add Security Headers **47**
Disable Forced https **48**
Add CORS Policies **49**
Make a Pull Request **50**
Planning **51**
Story 1: Containerize Your Microservice Using Docker 51
Story 2: Deploy Your Docker Image to Kubernetes 52
Story 3: Create a CD Pipeline to Automate Deployment to Kubernetes 53
Pick Up the First Story **54**
Create a Dockerfile **55**
Create a Docker Image **57**
Make a Pull Request **58**
Pick Up the Next Story **59**
Deploy to Kubernetes **60**
Create Manifests **61**
Pick Up the Next Story **64**
Overview and Setup **65**
Create the Lint Task **67**
Create a Test Task **70**
Add the Test Task to the Pipeline **76**
Start the Pipeline **79**
Create a Build Task **81**
Add the Deploy Task to the Pipeline **85**
Apply Changes and Run the Pipeline **89**

Project Overview

You have been asked by the customer account manager at your company to develop an account microservice to keep track of the customers on your e-commerce website. Since it is a microservice, it is expected to have a well-formed REST API that other microservices can call. This service initially needs to create, read, update, delete, and list customers.

You have also been told that someone else has started on this task. They have already developed the database model and a Python Flask-based REST API with an endpoint to create a customer account. You just need to plan to add the REST APIs to read, update, delete, and list accounts. Since you will be working in an online lab environment, you will need to plan your work to get that environment ready for development.

Create a GitHub Repository

Create a new GitHub repository for your project, called devops-capstone-project, from the template provided by the URL below:

Create a User Story Template

Create an issue template in GitHub for your project's GitHub repository. Ensure the template includes the components listed below. You may want to copy, paste, then edit this text because it also contains the correct markdown syntax you will need for the template.

As a [role]
I need [function]
So that [benefit]

Details and Assumptions
    * [document what you know]
Acceptance Criteria
    gherkin
    Given [some context]
    When [certain action is taken]
    Then [the outcome of action is observed]

Ensure that you have a new folder in your repository called .github/ISSUE_TEMPLATES. This folder will contain your new user story template.

Assemble Your Product Backlog

Create seven user stories in your Zenhub kanban board, one for each of the following steps of your project:

Set up the development environment
Read an account from the service
Update an account in the service
Delete an account from the service
List all accounts in the service
Containerize your microservice using Docker
Deploy your Docker image to Kubernetes

Triage New Issues

Determine which user stories you will work on immediately and move them from the New Issues pipeline to the Product Backlog pipeline.

Move the remaining stories from New Issues into the Ice box as you will work on them later.

Refine Your Product Backlog

Make sure that all the stories in the Product Backlog have sufficient details to be considered “sprint ready.” Pay special attention to the Acceptance Criteria to be sure that you have defined the definition of “done.”

#1

As a developer
I need development environment to work on tasks
So that tasks can be done

Details and Assumptions
* [document what you know]

Acceptance Criteria
gherkin
Given I have project requirements
When I work on tasks
Then I able to do development

#2

As a user
I need info about a customer
So that I can read customer data

Details and Assumptions
* [document what you know]

Acceptance Criteria
gherkin
Given Need to read customer record
When Customer info required
Then user able to access the customer record

#3

As a user
I need update info about a customer
So that I can update customer data

Details and Assumptions
* [document what you know]

Acceptance Criteria
gherkin
Given Need to update customer record
When Customer info need change
Then user able to change the customer record

#4

As a user
I need delete info about a customer
So that I can delete customer data

Details and Assumptions
* [document what you know]

Acceptance Criteria
gherkin
Given Need to delete customer record
When Customer info need to remove
Then user able to delete the customer record

#5

As a user
I need access all customer records
So that I can list all customers data

Details and Assumptions
* [document what you know]

Acceptance Criteria
gherkin
Given Need to access customer list

When All customers info need
Then user able to list the customer records

Build First Sprint from Product Backlog

Set up sprints in Zenhub and make the sprints one week in duration.

Rename the three sprints that Zenhub creates as: Sprint 1, Sprint 2, and Sprint 3, respectively. This will make it easier for you to track and others to grade.

Open the first story from the top of the Product Backlog and assign estimated story points to it. For now, use the scale 3, 5, 8, 13 = S, M, L, XL.

Assign the current story to Sprint 1.

Close the current story and move it from the Product Backlog to the Sprint Backlog, being careful to preserve its ranked order. For example, the story at the top of the Product Backlog would remain the top story in the Sprint Backlog.

Repeat steps 3 through 5 for the remaining four stories in the Product Backlog, preserving their ranked order.

Initialize Development Environment

Because the Cloud IDE with OpenShift environment is ephemeral, it may be deleted at any time. The next time you come into the lab, a new environment may be created. Unfortunately, this means that you will need to initialize your development environment every time it is recreated. This shouldn't happen too often as the environment can last for several days at a time but when it is removed, this is the procedure to recreate it.

Overview

Each time you need to set up your lab development environment you will need to run three commands.

Each command will be explained in further detail, one at a time, in the following section.

{your_github_account} represents your GitHub account username.

The commands include:

```
git clone https://github.com/sf-co/devops-capstone-project.git
cd devops-capstone-project
bash ./bin/setup.sh
```

Now, let's discuss each of these commands and explain what needs to be done.

Task Details

Initialize your environment using the following steps:

Open a terminal with Terminal -> New Terminal if one is not open already.

Next, use the export GITHUB_ACCOUNT= command to export an environment variable that contains the name of your GitHub account.

Note: Substitute your real GitHub account for the {your_github_account} placeholder below:

```
export GITHUB_ACCOUNT=sf-co
```

Then use the following commands to clone your repository, change into the devops-capstone-project directory, and execute the ./bin/setup.sh command.

```
git clone https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git
cd devops-capstone-project
```

bash ./bin/setup.sh

You should see the follow at the end of the setup execution:

capstone_setup_complete

Finally, use the exit command to close the current terminal. The environment will not be fully active until you open a new terminal in the next step.

Validate

In order to validate that your environment is working correctly, you must open a new terminal because the Python virtual environment will only activate when a new terminal is created. You should have ended the previous task by using the exit command to exit the terminal.

Open a terminal with Terminal -> New Terminal and check that everything worked correctly by using the which python command:

Your prompt should look like this:

Setup Prompt

Check which Python you are using:

which python

Check the Python version:

python --version

You should get back some patch level of Python 3.9:

Python Version

This completes the setup of the development environment. Anytime your environment is recreated, you will need to follow this procedure.

You are now ready to start working.

Project Overview

You have been asked by the customer account manager at your company to develop an Account microservice to keep track of the customers on your e-commerce website. In the Agile Planning lab, you created a plan to do just that. Since it is a microservice, it is expected to have a well-formed REST API that other microservices can call. This service initially needs the ability to create, read, update, delete, and list customers.

You have also been informed that someone else has started on this task and has already developed the database model and a Python Flask-based REST API with an endpoint to create a customer account. You should already have this code from when you created your own repo for the Agile Planning lab. If you have not completed that lab, please stop and do so now.

REST API

RESTful API Endpoints

Action	Method	Return code	Body	URL Endpoint
List	GET	200_OK	Array of accounts [{...}]	GET /accounts
Create	POST	201_CREATED	An account as json {...}	POST /accounts
Read	GET	200_OK	An account as json {...}	GET /accounts/{id}
Update	PUT	200_OK	An account as json {...}	PUT /accounts/{id}
Delete	DELETE	204_NO_CONTENT	""	DELETE /accounts/{id}

HTTP Status Codes
Here are some other HTTP status codes that you will need for this lab:

Code	Status	Description
200	HTTP_200_OK	Success
201	HTTP_201_CREATED	The requested resource has been created
204	HTTP_204_NO_CONTENT	There is no further content
404	HTTP_404_NOT_FOUND	Could not find the resource requested
405	HTTP_405_METHOD_NOT_ALLOWED	Invalid HTTP method used on an endpoint
409	HTTP_409_CONFLICT	There is a conflict with your request

All of these codes are defined in service/common/status.py and are already imported for your use.

Implement First User Story

Take the story titled "Set up the development environment" from the top of the Sprint Backlog and move it to the In Progress column and assign it to yourself.

Change into the project directory devops-capstone-project

Create a new branch called dev-setup to begin working on the story.

git checkout -b dev-setup

Edit the setup.cfg file and add the flags in the example above, under the [nosetests] stanza.

[nosetests]
verbosity=2
with-spec=1
spec-color=1
with-coverage=1
cover-erase=1
cover-package=service

Run nosetests just to be sure that the file is being read correctly and does not cause an error. If it does, fix it.

Use the git commit -am command to commit your changes with the message "added nose arguments"``, and the git push` command to push those changes to your repository.

Note: You will be prompted to set up your git user and email the first time you push:

git config --local user.name "Ali Imran"
git config --local user.email sw@gmail.com

git commit -am "added nose arguments"
git push --set-upstream origin dev-setup

Create a pull request on GitHub to merge your changes into the main branch, and, since there is no one else on your team, accept the pull request, merge it, and delete the branch.

Finally, go to your kanban board and move your story into the Done column to show it has been completed.

Now that you have established your setup.cfg file, you can get better test output from Nose without including the parameters in your command.

Create a REST API with Flask

Task 1: Select the Next Story to Work On

Go to your Zenhub kanban board, take the next story from the top of the Sprint Backlog, and move it to the In Progress column.

Open the story and assign it to yourself.

Read the story to understand what you need to implement.

Task 2: Create a Branch

Since you are working in branches, you must pull the latest changes from the main branch to stay up to date as you merge each story.

The steps are:

git checkout main
git pull
git branch -d {old_branch_name}
git checkout -b read-account

This will switch to the main branch, pull the latest changes, delete your old branch, and create a new branch.

Task 3: Write a Test Case and Watch It Fail

Following test driven development, you write a test case to assert that the code you are about to write will have the correct behavior as outlined in the acceptance criteria of the story.

For example if the story is Read an account from the service, then you might do the following:

Open test_routes.py in IDE

Note: To open in file explorer go to the location:

devops-capstone-project/tests/test_routes.py

Create a test case called test_read_an_account(self).
Make a self.client.post() call to accounts to create a new account passing in some account data.
Get back the account id that was generated from the json.
Make a self.client.get() call to /accounts/{id} passing in that account id.

Assert that the return code was HTTP_200_OK.
Check the json that was returned and assert that it is equal to the data that you sent.
Run nosetests and watch it fail because there is no code yet.

This is a complete test case for read an account:

```
    def test_get_account(self):
        """It should Read a single Account"""
        account = self._create_accounts(1)[0]
        resp = self.client.get(
            f"{BASE_URL}/{account.id}", content_type="application/json"
        )
        self.assertEqual(resp.status_code, status.HTTP_200_OK)
        data = resp.get_json()
        self.assertEqual(data["name"], account.name)
```

Task 4: Write the Code to Make the Test Case Pass

Once you have a test case, you can begin to write the code to make it pass. Assuming that you are working on the "read an account" story, you might do the following:

Open routes.py in IDE

Note: To open in file explorer go to the location:

devops-capstone-project/service/routes.py

Create a Flask route that responds to the GET method for the endpoint /accounts/<id>.
Create a function called read_account(id) to hold the implementation.
Call the Account.find() which will return an account by id.
Abort with a return code HTTP_404_NOT_FOUND if the account was not found.
Call the serialize() method on an account to serialize it to a Python dictionary.
Send the serialized data and a return code of HTTP_200_OK back to the caller.
Run nosetests until all of the tests are green, which means they passed.

This is a complete REST API implementation for reading an account:

```
    ######################################################################
    # READ AN ACCOUNT
    ######################################################################
    @app.route("/accounts/<int:account_id>", methods=["GET"])
    def get_accounts(account_id):
        """
        Reads an Account
```

```
    This endpoint will read an Account based the account_id that is requested
    """
    app.logger.info("Request to read an Account with id: %s", account_id)
    account = Account.find(account_id)
    if not account:
        abort(status.HTTP_404_NOT_FOUND, f"Account with id [{account_id}] could not be
found.")
    return account.serialize(), status.HTTP_200_OK
```

Task 5: Check Your Code Coverage

You must maintain code coverage of 95% or greater. You will not achieve this by only testing the happy paths. The test case you wrote probably did not test for an account that was not found, so you will need to write another test case that reads an account with an account id that does not exist. This should get your test coverage back up to where it needs to be.

Create a test case called test_account_not_found(self).

Make a self.client.get() call to /accounts/{id} passing in 0 as the account id.
Assert that the return code was HTTP_404_NOT_FOUND.
Run nosetests and fix the code in routes.py until it passes.

Task 6: Make a Pull Request

Now you are ready to push the code back to Github and make a pull request.

Run nosetests to make sure that all of the tests pass. If they are not, fix them.
Use git commit -am "account read" to commit your changes.
Use git push --set-upstream origin read-account to push your changes to GitHub.
Create a pull request on GitHub to merge your changes into the main branch.
Since there is no one else on your team, accept the pull request, merge it, and delete the branch.

Task 7: Update the Kanban Board

Your final task is to update the kanban board to let everyone else on the team know that you are done.

Move your story into the Done column to show it has been completed.

Now, go back to Task 1 and work on the next story until all of the stories are implemented. On the following page, you will find the remaining hints and solutions. You may refer to them while writing your code.

Run the REST service

In the bash terminal, use the flask db-create command to refresh the database.

flask db-create

Use the make run command to start the service with the new database.

make run

Launch the application in a web browser by pressing the Launch Account Service button below:

Launch Account Service

Results

You should get back the following web page:

capstone_service_running

You are now ready to test your running service.

Sprint Review

Demo Create Account

Enter the following command to create an account:

```
curl -i -X POST http://127.0.0.1:5000/accounts \
-H "Content-Type: application/json" \
-d '{"name":"John Doe","email":"john@doe.com","address":"123 Main
St.","phone_number":"555-1212"}'
```

Check the account id that was returned. It should have been 1 but if it was not, substitute the account id that was returned for all of the remaining calls.

Demo List All Accounts

Enter the following command to list all accounts:

```
curl -i -X GET http://127.0.0.1:5000/accounts
```

Take a screenshot of the output and save the screenshot as rest-list-done.jpg (or rest-list-done.png).

Demo Read an Account

Enter the following command to read the account:

```
curl -i -X GET http://127.0.0.1:5000/accounts/1
```

Demo Update an Account

Enter the following command to update the account:

```
curl -i -X PUT http://127.0.0.1:5000/accounts/1 \
```

```
-H "Content-Type: application/json" \
-d '{"name":"John Doe","email":"john@doe.com","address":"123 Main
St.","phone_number":"555-1111"}'
```

Note: You are sending a new phone number. Check that the phone number returned is 555-1111.

Demo Delete an Account

Enter the following command to delete the account:

```
curl -i -X DELETE http://127.0.0.1:5000/accounts/1
```

You have completed the demo of the REST calls you implemented, and the product owner has agreed that they are all done as expected.

Move Stories to Closed

Planning

New Requirements

Management has been looking for ways to increase developer productivity and has noticed that developers spend a lot of time checking that all the tests pass before approving each pull request. Management has decided it is time to automate this task by implementing continuous integration (CI) using GitHub Actions.

There have also been many stories in the news about security breaches and exploits, and management is concerned that about the security of your microservice. In an effort to be proactive, they have decided that you need to add defensive security measures to your microservice in the form of security headers and cross-origin resource sharing (CORS) policies.

To satisfy these requirements, you need to add two new stories to your Sprint Backlog.

Need the ability to automate continuous integration checks
Need to add security headers and CORS policies

Then you will make a sprint plan from them.

Story 1: Need the Ability to Automate Continuous Integration Checks

Create the following story about CI automation in the Product Backlog column of your Zenhub kanban board:

Title: Need the ability to automate continuous integration checks
As a Developer
I need automation to build and test every pull request
So that I do not have to rely on manual testing of each request, which is time-consuming
Assumptions
* GitHub Actions will be used for the automation workflow
* The workflow must include code linting and testing
* The Docker image should be postgres:alpine for the database
* A GitHub Actions badge should be added to the README.md to reflect the build status
Acceptance Criteria
```gherkin
Given code is ready to be merged
When a pull request is created
```

````
Then GitHub Actions should run linting and unit tests
And the badge should show that the build is passing
```
````

Since this story adds no visible stakeholder value and is primarily to make development more efficient, assign this story a technical debt label.

Give the story an estimate. Remember the values are Small = 3, Medium = 5, Large = 8, and Extra Large = 13.

Next, assign the story to Sprint 2.

Finally, move the story to the top of the Sprint Backlog.

Story 2: Need to Add Security Headers and CORS Policies

Create the following story about security enhancements in the Product Backlog column of your Zenhub kanban board:

````
Title: Need to add security headers and CORS policies
**As a** service provider
**I need** my service to use security headers and CORS policies
**So that** my web site is not vulnerable to CORS attacks
#### Assumptions
* Flask-Talisman will be used for security headers
* Flask-Cors will be used to establish cross-origin resource sharing (CORS) policies
#### Acceptance Criteria
```gherkin
Given the site is secured
When a REST API request is made
Then secure headers and a CORS policy should be returned
```
````

This story will make the microservice more secure, and better security adds stakeholder value. Assign the appropriate label to this story.

Give the story an estimate. Remember the values are Small = 3, Medium = 5, Large = 8, and Extra Large = 13.

Next, assign the story to Sprint 2.

Finally, move the the story to the Sprint Backlog, ranking it second.
````

Initialize Development Environment

Because the Cloud IDE with Docker environment is ephemeral, it may be deleted at any time. The next time you come into the lab, a new environment may be created. Unfortunately, this means that you will need to initialize your development environment every time it is recreated. This should not happen too often as the environment can last for several days at a time but when it is removed, this is the procedure to recreate it.

Overview

Each time you need to set up your lab development environment, you will need to run three commands.

Each command will be explained in further detail, one at a time, in the following section.

{your_github_account} represents your GitHub account username.

The commands include:

git clone https://github.com/{your_github_account}/devops-capstone-project.git
cd devops-capstone-project
bash ./bin/setup.sh

Now, let's discuss each of these commands and explain what needs to be done.

Task Details

Initialize your environment using the following steps:

Open a terminal with Terminal -> New Terminal if one is not open already.

Next, use the export GITHUB_ACCOUNT= command to export an environment variable that contains the name of your GitHub account.

Note: Substitute your GitHub username for the {your_github_account} place holder below:

export GITHUB_ACCOUNT=sf-co

Then use the following commands to clone your repository, change into the devops-capstone-project directory, and execute the ./bin/setup.sh command.

git clone https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git

cd devops-capstone-project
bash ./bin/setup.sh
Copied!Executed!
You should see the following at the end of the setup execution:

capstone_setup_complete

Finally, use the exit command to close the current terminal. The environment will not be fully active until you open a new terminal in the next step.

Validate

In order to validate that your environment is working correctly, you must open a new terminal because the Python virtual environment will only activate when a new terminal is created. You should have ended the previous task by using the exit command to exit the terminal.

Open a terminal with Terminal -> New Terminal and check that everything worked correctly by using the which python command:

Your prompt should look like this:

Setup Prompt

Check which Python you are using:

which python

You should get back:

Which Python

Check the Python version:

python --version

You should get back some patch level of Python 3.9:

Python Version

This completes the setup of the development environment. Anytime your environment is recreated, you will need to follow this procedure.

You are now ready to start working.

Pick Up the First Story

Go to your Zenhub kanban board and take the first story from the top of the Sprint Backlog. It should be titled: "Need the ability to automate continuous integration checks“.

Move the story to In Progress.

Open the story and assign it to yourself.

Read the contents of the story.

Results

The story should look like this:

Need the ability to automate continuous integration checks
As a Developer
I need automation to build and test every pull request
So that I do not have to rely on manual testing of each request, which is time-consuming

Assumptions

GitHub Actions will be used for the automation workflow
The workflow must include code linting and testing
The Docker image should be postgres:alpine for the database
A GitHub Actions badge should be added to the README.md to reflect the build status

Acceptance Criteria

Given code is ready to be merged
When a pull request is created
Then GitHub Actions should run linting and unit tests
And the badge should show that the build is passing

Create a Workflow

Take the first story "Need the ability to automate continuous integration checks" from the Sprint Backlog and move it into In Progress and assign it to yourself.

Create a new branch named add-ci-build to work on in the development environment.

git checkout -b add-ci-build

Create a GitHub Actions workflow in the .github/workflows folder of your GitHub repository called ci-build.yaml

Give the workflow the name: CI Build.

Set up the on: statement to run this workflow on every push and pull_request to the main branch of the repository.

Validate

Your workflow trigger definition should look like this:

Click here to check your answer.

```
name: CI Build
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

Create a Job

Create a job called build.

Here is a template:

```
jobs:
  build:
```

Specify that the build job runs on ubuntu:latest.

Define a container: for the build job to run it, that uses a python:3.9-slim image.

Validate

Your job section should look like this:

Click here to check your answer.

```
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
```

You are now ready to create a database service.

Define Required Services

This is the Docker command from your Makefile that is used for testing while developing locally. You can use the parameters from it to define a Postgres service for your workflow.

```
docker run -d --name postgres -p 5432:5432 \
      -e POSTGRES_PASSWORD=pgs3cr3t \
      -v postgres:/var/lib/postgresql/data \
      postgres:alpine
```

Create a services: section in your workflow file at the same level of indentation as the container: section.

Here is a template:

```
jobs:
  build:
    runs-on: {runner_name}
    container: {image_name}
    services:
```

Under the services: section, define a service called postgres:.

Here is a template:

```
jobs:
  build:
    runs-on: {runner_name}
    container: {image_name}
    services:
      postgres:
```

Create an image: tag under the Postgres service with a value of postgres:alpine.

Here is a template:

```
jobs:
  build:
    runs-on: {runner_name}
    container: {image_name}
    services:
      postgres:
```

```
      image: {image name here...}
```

Create a ports: tag under the Postgres service that is listening on port 5432 (which is the default Postgres port).

Here is a template:

```
jobs:
  build:
    runs-on: {runner_name}
    container: {image_name}
    services:
      postgres:
        image: {image name here...}
        ports:
          - {port}:{port}
```

Add an env: environment variable tag and define values for POSTGRES_PASSWORD and POSTGRES_DB. Use the values pgs3cr3t and testdb, respectively.

Here is a template:

```
jobs:
  build:
    runs-on: {runner_name}
    container: {image_name}
    services:
      postgres:
        image: {image name here...}
        ports:
          - {port}:{port}
        env:
          POSTGRES_PASSWORD: {password here...}
          POSTGRES_DB: {database name here...}
```

Add an options: tag to check for a health-cmd of pg_isready, health-interval of 10s, health-timeout of 5s, and health-retries of 5.

Validate

Your service definition should look like this:

Click here to check your answer.

```
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
    services:
      postgres:
        image: postgres:alpine
        ports:
          - 5432:5432
        env:
          POSTGRES_PASSWORD: pgs3cr3t
          POSTGRES_DB: testdb
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
```

You are now ready to start adding steps.

Check Out Code and Install Dependencies

You are now ready to define the steps for your workflow. The first steps are to check out the code and install the dependencies. For checking out code, you will use an action from the GitHub Actions Marketplace.

To install the dependencies, you can use the same statements as if you were doing this locally on your computer.

Reference

You can use the following commands to install the Python package dependencies in your workflow steps.

```
python -m pip install --upgrade pip wheel
pip install -r requirements.txt
```

Your Task

Add a step named Checkout to check out the code using the actions/checkout@v2 action. Substitute the step name and action name from the instructions.

```
  steps:
    - name: {checkout step name here...}
      uses: {action name here...}
```

Add a step named Install dependencies that upgrades pip and wheel and then uses the pip command to install the Python packages from the requirements.txt file.

Substitute the step name and command name from the instructions.

```
  steps:
    - name: {checkout step name here...}
      uses: {action name here...}
    - name: {install step name here...}
      run: |
       {commands to install dependencies here...}
```

Validate

Your first two step definitions should look like this:

jobs:

```
  build:
    services:
    # ...
    steps:
      - name: Checkout
        uses: actions/checkout@v2
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip wheel
          pip install -r requirements.txt
```

You are now ready to move on to the next step.

Add Linting

In this step, you will add linting, which is checking your code for syntactical and stylistic issues. Some examples are line spacing, using spaces or tabs for indentation, locating uninitialized or undefined variables, and missing parentheses.

It is always a good idea to add quality checks to your CI pipeline. This is especially true if you are working on an open source project with many different contributors. This makes sure that everyone who contributes is following the same style guidelines.

Now, you will use flake8 to lint the source code.

Note: The flake8 library was installed as a dependency in the requirements.txt file.

Reference

You can use the following commands to run the flake8 linter in your workflow steps.

```
flake8 service --count --select=E9,F63,F7,F82 --show-source --statistics
flake8 service --count --max-complexity=10 --max-line-length=127 --statistics
```

Note: You should run these commands on your code before you add them to your workflow to ensure that your code passes the tests. You can use the make lint command to do this.

Your Task

Add a new step named Lint with flake8 after the Install dependencies step that runs the flake8 commands from the above reference.

Substitute the step name and command name from the instructions.

```
  steps:
    {... other steps here ...}
    - name: {step name here...}
      run: |
       {commands to run flake8 here...}
```

Validate

Your lint step definition should look like this:

Click here to check your answer.

```
jobs:
  build:
    services:
    # ...
    steps:
      # ...previous steps here
      - name: Lint with flake8
        run: |
          flake8 service --count --select=E9,F63,F7,F82 --show-source --statistics
          flake8 service --count --max-complexity=10 --max-line-length=127 --statistics
```

You are now ready to move on to the next step.

Add Unit Testing

To satisfy the testing requirements, you will use Nose in this step to unit test the source code. Nose is configured via the included setup.cfg file to automatically include the flags --with-spec and --spec-color so that red-green-refactor is meaningful. If you are in a command shell that supports colors, passing tests will be green and failing tests will be red.

Nose is also configured to automatically run the coverage tool, and you should see a percentage of coverage report at the end of your tests.

Reference

You can use the following commands to run the nosetests in your workflow steps.

nosetests -v --with-spec --spec-color --with-coverage --cover-package=service

Note: You should run these commands on your code before you add them to your workflow to ensure that your code passes the tests. You can use the make test command to do this.

Your Task

Add a new step named Run unit tests with Nose after the Lint with flake8 step that runs the nosetests commands from the above reference.

Substitute the step name and command name from the instructions. Since you are running a single command, you do not have to use the pipe | operator with run.

```
  steps:
    {... other steps here ...}
    - name: {step name here...}
      run: {command to run nosetests here...}
```

Use the env: tag to add an environment variable named DATABASE_URI that will configure the tests to use the PostgreSQL database that you created in the service: section.

Substitute the step name and command name from the instructions. Since you are running a single command, you do not have to use the pipe | operator with run.

```
  steps:
    {... other steps here ...}
    - name: {step name here...}
      run: {command to run nosetests here...}
      env:
        DATABASE_URI: "postgresql://postgres:{password}@{service}:{port}/{database}"
```

Validate

Your test step definition should look like this:

```
jobs:
  build:
    services:
    # ...
    steps:
      # ...previous steps here
      - name: Run unit tests with nose
        run: nosetests
        env:
          DATABASE_URI: "postgresql://postgres:pgs3cr3t@postgres:5432/testdb"
```

That completes your workflow definition.

Check your final work

Try and run the workflow that you put together yourself. If anything goes wrong, here is the complete solution so that you can check how it may differ from yours.

```
name: CI Build
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
    services:
      postgres:
        image: postgres:alpine
        ports:
          - 5432:5432
        env:
          POSTGRES_PASSWORD: pgs3cr3t
          POSTGRES_DB: testdb
        options: >-
```

```
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
steps:
  - name: Checkout
    uses: actions/checkout@v2
  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip wheel
      pip install -r requirements.txt
  - name: Lint with flake8
    run: |
      flake8 service --count --select=E9,F63,F7,F82 --show-source --statistics
      flake8 service --count --max-complexity=10 --max-line-length=127 --statistics
  - name: Run unit tests with nose
    run: nosetests
    env:
      DATABASE_URI: "postgresql://postgres:pgs3cr3t@postgres:5432/testdb"
```

Create a Badge

One of the last requirements listed in the story under Assumptions is:

A GitHub Actions badge should be added the README.md to reflect the build status
Now it's time to add that badge.

The format of a GitHub Actions badge looks like this:

```
![Build
Status](https://github.com/<OWNER>/<REPOSITORY>/actions/workflows/<WORKFLOW_FILE
>/badge.svg)
```

Copied!
Where <OWNER> is the account name or organization name of the repository.
<REPOSITORY> is the name of the repository, which you already know is devops-capstone-project.
And <WORKFLOW_FILE> is the name of the GitHub Actions workflow file that the badge represents, which you already know is ci-build.yaml.
Your Task
Edit the README.md file.

On a line below the title and separated from the title by a blank line above and below, add the following markdown:

```
![Build
Status](https://github.com/<OWNER>/devops-capstone-project/actions/workflows/ci-build.yaml/
badge.svg)
```

Where <OWNER> is the name of your GitHub account.
Save the file and commit your changes using the message Added badge for GitHub Actions.
Once you make a pull request, the badge will show everyone the status of the build.

Make a Pull Request

Now that you have completed your GitHub Action, you are ready to commit your changes and push code to your GitHub repository and make a pull request.

Your Task

Commit your changes locally in the development environment with the message "completed ci build".

git commit -am "{message here}"

git commit -am "completed ci build"

Push your local changes to a remote branch.

git push
git push --set-upstream origin add-ci-build

Go to GitHub and make a pull request, which should kick off the GitHub Action that you just wrote.

Pick Up the Next Story

The first thing you need to do is to go to Zenhub to get a story to work on. Take the next story from the top of the Sprint Backlog, move it to the In Progress column, assign it to yourself, and read the contents.

Your Task

Go to your kanban board and take the next story from the top of the Sprint Backlog column. It should be titled: "Need to add security headers and CORS policies".

Move the story to In Progress.

Open the story and assign it to yourself.

Read the contents of the story.

Results

The story should look like this:

Need to add security headers and CORS policies
As a service provider
I need my service to use security headers and CORS policies
So that my web site is not vulnerable to CORS attacks

Assumptions

Flask-Talisman will be used for security headers
Flask-Cors will be used to establish cross-origin resource sharing (CORS) policies
Acceptance Criteria

Given the site is secured
When a REST API request is made
Then secure headers and a CORS policy should be returned

You are now ready to begin working on your story.

Observe the Current Behavior

In reading your story, you see that the first assumption is:

Flask-Talisman will be used for security headers
This will be the first change you make. But before you do, it's a good idea to observe the current behavior so that you have something to compare to once you add Talisman.

Your Task

Open a terminal and make sure that you are in the /home/project/devops-capstone-project folder.

cd /home/project/devops-capstone-project

Use the honcho start command to start your microservice running in the terminal listening on port 5000.

honcho start

Open another terminal (Terminal -> New Terminal) and use the following curl command to see the headers that are being returned.

curl -I localhost:5000

Go back to the first terminal and use Ctrl+C to stop the running server.

Results
You should see output similar to the following:

HTTP/1.1 200 OK
Server: gunicorn
Date: Thu, 13 Oct 2022 12:07:32 GMT
Connection: close
Content-Type: application/json
Content-Length: 52

Make a mental note of this output. Once you add Talisman, you will see security headers included.

Write a Test Case

In reading your story, you see that the first assumption is:

Flask-Talisman will be used for security headers

In keeping with test driven development (TDD) practices, you want to write a test case to test for the behavior that you are about to implement. While the test case will fail at first, it will also let you know when the implementation is successful.

Flask-Talisman forces your REST API clients to use the HTTPS protocol. This means that if you want to test this behavior, your test case must use https:// in order to work. To get the Flask test client to use https you can use the environ_overrides attribute.

Your Task

You will be working with the ./tests/test_routes.py file:

Open test_routes.py in IDE

Use the git checkout -b command to create a new branch to work on in the development environment.

Run nosetests and make sure that all of the test cases are passing. Fix any that fail before proceeding.

Edit tests/test_routes.py and add the following line of code toward the top of the file after the line that defines the BASE_URL global variable.

HTTPS_ENVIRON = {'wsgi.url_scheme': 'https'}

Write a test case that calls the root URL "/" passing in environ_overrides=HTTPS_ENVIRON as a parameter and asserting the presence of the following headers and their values in the output:

'X-Frame-Options': 'SAMEORIGIN'
'X-XSS-Protection': '1; mode=block'
'X-Content-Type-Options': 'nosniff'
'Content-Security-Policy': 'default-src \'self\'; object-src \'none\'
'Referrer-Policy': 'strict-origin-when-cross-origin'

Run nosetests and watch that test case fail with an AssertionError because it cannot find the headers. You are now ready to write the code to add the missing security headers.

Add Security Headers

In this step, you will add Talisman to your requirements file and add code to use Talisman in your microservice.

You will be working with the ./service/__init__.py file:

Open __init__.py in IDE

Your Task

Add Flask-Talisman to your requirements.txt file.

Use pip to install the new requirements using -r requirements.txt.

Open the service/__init__.py file and import the Talisman class from flask_talisman.

from flask_talisman import Talisman

Next, after the Flask app is created, create an instance of the Talisman class called talisman passing in the Flask app to the class constructor.

talisman = Talisman(app)

Run nosetests tests/test_routes.py to run only the tests for the routes and observe the output.

Results

All of your test cases have failed except for the new one that tests for the security headers. You will fix these errors in the next exercise.

Disable Forced https

By default, Talisman will force all requests to your REST API to use the https:// protocol. This is a good thing, except perhaps when testing. Luckily, Talisman gives you a way to turn this behavior on and off.

In this exercise, you will disable forced https by setting force_https = False on the talisman instance from your service.

Your Task

Open tests/test_routes.py and import talisman from the service package.

Update the setUpClass() method by adding the line: talisman.force_https = False

Run nosetests tests/test_routes.py again and all of the tests should now pass.

Finally, when all the tests have passed, use the git commit -am to commit your changes with the message Added security headers.

Make sure that all of the tests pass before moving on to the next exercise.

Add CORS Policies

Going back to the story that you are working on, you see that the second assumption is

Flask-Cors will be used to establish cross-origin resource sharing (CORS) policies
This will be the next change you make.

Your Task

Write a test case that calls the root URL "/" passing in environ_overrides=HTTPS_ENVIRON as a parameter and asserting the presence of the header Access-Control-Allow-Origin: * in the output.

This is the test case for checking that the CORS headers is present:

```
    def test_cors_security(self):
        """It should return a CORS header"""
        response = self.client.get('/', environ_overrides=HTTPS_ENVIRON)
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        # Check for the CORS header
        self.assertEqual(response.headers.get('Access-Control-Allow-Origin'), '*')
```

Run nosetests and watch that test case fail with an AssertionError because it cannot find the headers.

Add Flask-Cors to your requirements.txt file.

Use pip to install the new requirements using -r requirements.txt.

Open the service/__init__.py file and import the CORS class from flask_cors.

```
from flask_cors import CORS
```

Next, after the Talisman class is created, create the CORS class passing in the Flask app.

```
CORS(app)
```

Run nosetests to run all of the unit tests and observe the output.

Finally, when all the tests have passed, use git commit -am to commit your changes with the message Added CORS headers.

Results All of your test cases should pass.

Make a Pull Request

Now that you have completed your GitHub Action, you are ready to commit your changes, push code to your GitHub repository, and make a pull request.

Your Task
Use git status to make sure that you have committed your changes locally in the development environment.

Push your local changes to a remote branch.

Make a pull request, which should kick off the GitHub Action that is now enabled on your repository.

Evidence

Open the service/__init__.py file.
Take a screenshot and save it as security-code-done.jpg (or security-code-done.png).
Take a screen shot of the output from Exercise 7 and save it as security-headers-done.jpg (or security-headers-done.png).
Move your story to the Done column on your kanban board.
Take a screenshot of your kanban board and save it as security-kanban-done.jpg (or security-kanban-done.png).

Planning

Management has been very pleased with the changes you have been making. It's now time to create a sprint plan to implement the last two stories in your Product Backlog, which are "Containerize your microservice using Docker" and "Deploy your Docker image to Kubernetes.“

One more thing. There is a new requirement. You did such a great job automating the CI pipeline with GitHub Actions that all of the developers seem much happier because of it. Management has decided that if a little automation is good, then more automation would be better. They would like you to automate the deployment to Kubernetes using Tekton once you have figured out how to do it manually.

To satisfy these requirements, you will need to move the two existing stories from the Product Backlog and add one new story to your Sprint Backlog.

Containerize your microservice using Docker
Deploy your Docker image to Kubernetes
Create a CD pipeline to automate deployment to Kubernetes
Then you will make a sprint plan from them.

Story 1: Containerize Your Microservice Using Docker

This story should already be in your Product Backlog from the original sprint planning session. It's a good idea to review and confirm that you have a "sprint ready" story.

Check that the "Containerize your microservice using Docker" story in the Product Backlog column of your Zenhub kanban board looks something like this:

Note: You can cut-n-paste the markdown text below into your story:

Title: Containerize your microservice using Docker
As a developer
I need to containerize my microservice using Docker
So that I can deploy it easily with all of its dependencies
Assumptions
* Create a `Dockerfile` for repeatable builds
* Use a `Python:3.9-slim` image as the base
* It must install all of the Python requirements
* It should not run as `root`
* It should use the `gunicorn` wsgi server as an entry point
Acceptance Criteria

Given the Docker image named accounts has been created
When I use `docker run accounts`
Then I should see the accounts service running in Docker

Since this story adds no visible stakeholder value and is primarily to make development more efficient, assign this story a label of technical debt.

Give the story an estimate. Remember the values are Small = 3, Medium = 5, Large = 8, and Extra Large = 13.

Next, assign the story to Sprint 3.

Finally, move the story to the top of the Sprint Backlog.

Story 2: Deploy Your Docker Image to Kubernetes

This story should already be in your Product Backlog from the original sprint planning session. It's a good idea to review and confirm that you have a "sprint ready" story.

Check that the "Deploy your Docker image to Kubernetes" story in the Product Backlog column of your Zenhub kanban board looks something like this:

Note: You can cut-n-paste the markdown text below into your story:

Title: Deploy your Docker image to Kubernetes
As a service provider
I need my service to run on Kubernetes
So that I can easily scale and manage the service
Assumptions
* Kubernetes manifests will be created in yaml format
* These manifests could be useful to create a CD pipeline
* The actual deployment will be to OpenShift
Acceptance Criteria
Given the Kubernetes manifests have been created
When I use the oc command to apply the manifests
Then the service should be deployed and run in Kubernetes

This story will make the service scalable and perform better and thus give the users a better experience. It could be seen as an enhancement. Assign the appropriate label to this story.

Give the story an estimate. Remember the values are Small = 3, Medium = 5, Large = 8, and Extra Large = 13.

Next, assign the story to Sprint 3.

Finally, move the the story to the Sprint Backlog, ranking it second.

Story 3: Create a CD Pipeline to Automate Deployment to Kubernetes

In this story, you will create a CD pipeline using Tekton to deploy to OpenShift. It will reuse the Kubernetes manifest that you created in the "Deploy to Kubernetes" story. This is why you didn't simply use the command line interface to deploy the service in that story.

Create the following story about creating a CD pipeline in the Product Backlog column of your Zenhub kanban board:

Note: You can cut-n-paste the markdown text below into your story:

Title: Create a CD pipeline to automate deployment to Kubernetes
As a developer
I need to create a CD pipeline to automate deployment to Kubernetes
So that the developers are not wasting their time doing it manually
Assumptions
* Use Tekton to define the pipeline
* It should clone, lint, test, build, and deploy the service
* Deployment should be to OpenShift
* It can use a manual trigger for this MVP
Acceptance Criteria
Given the CD pipeline has been created
When I trigger the pipeline run
Then I should see the accounts service deployed to OpenShift

You could make the case that this story is an enhancement because deploys will be faster so customers get new features faster, or technical debt because it doesn't add enough customer value. It's up to you. Assign the appropriate label to this story.

Give the story an estimate. Remember the values are Small = 3, Medium = 5, Large = 8, and Extra Large = 13.

Next, assign the story to Sprint 3.
Finally, move the the story to the Sprint Backlog, ranking it third.
You are now ready to start the sprint.

Pick Up the First Story

Go to your Zenhub kanban board and take the first story from the top of the Sprint Backlog. It should be titled: "Containerize your microservice using Docker".

Move the story to In Progress.

Open the story and assign it to yourself.

Read the contents of the story.

Results

The story should look similar to this:

Containerize your microservice using Docker
As a developer

I need to containerize my microservice using Docker
So that I can deploy it easily with all of its dependencies

Assumptions

Create a Dockerfile for repeatable builds
Use a Python:3.9-slim image as the base
It must install all of the Python requirements
It should not run as root
It should use the gunicorn wsgi server as an entry point

Acceptance Criteria

Given the Docker image named accounts has been created
When I use `docker run accounts`
Then I should see the accounts service running in Docker

You are now ready to begin working on your story.

Create a Dockerfile

In reading your story you see that the assumptions state that you must create a Dockerfile with the following attributes:

Create a Dockerfile for repeatable builds
Use a Python:3.9-slim image as the base
It must install all of the Python requirements
It should not run as root
It should use the gunicorn wsgi server as an entry point
Let's take these in order.

Your Task

Change to your project directory: cd devops-capstone-project.

Use the git checkout -b add-docker command to create a new branch called add-docker to work on in the development environment.

Run nosetests and make sure that all of the test cases are passing. Fix any failing tests before proceeding.

In the root of the repository, create a file named: Dockerfile.

Edit the Dockerfile and start it FROM the python:3.9-slim image.

FROM python:3.9-slim

Establish a WORKDIR of /app, COPY the requirements.txt file into the working directory in the image, and RUN the pip command to install the requirements using the --no-cache-dir option to keep the image small.

```
# Create working folder and install dependencies
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

Copy the service package into the working directory of the same name in the image.

```
# Copy the application contents
COPY service/ ./service/
```

Create a non-root user called theia, change the ownership of the /app folder recursively to theia, and switch to the theia user.

Switch to a non-root user
RUN useradd --uid 1000 theia && chown -R theia /app
USER theia

Finally, EXPOSE port 8080 and create a CMD statement that runs: gunicorn --bind=0.0.0.0:8080 --log-level=info service:app

Run the service
EXPOSE 8080
CMD ["gunicorn", "--bind=0.0.0.0:8080", "--log-level=info", "service:app"]
Copied!
Results
You can check that your Dockerfile looks like the following:

FROM python:3.9-slim
Create working folder and install dependencies
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
Copy the application contents
COPY service/ ./service/
Switch to a non-root user
RUN useradd --uid 1000 theia && chown -R theia /app
USER theia
Run the service
EXPOSE 8080
CMD ["gunicorn", "--bind=0.0.0.0:8080", "--log-level=info", "service:app"]

Create a Docker Image

Open a terminal and use the docker build command to build a Docker image called accounts from the Dockerfile.

docker build -t accounts .

Use the docker run command to test that your image works properly. The PostgreSQL database is running in a Docker container named postgres so you will need to --link postgres and set the environment variable DATABASE_URI to point to it. You might also want to use the --rm flag to remove the container when it exists.

If it worked, you should see the message:

... [INFO] [__init__] Service initialized!

```
docker run --rm \
   --link postgresql \
   -p 8080:8080 \
   -e DATABASE_URI=postgresql://postgres:postgres@postgresql:5432/postgres \
   accounts
```

Check that your application is running by (1) clicking the Skills Network icon, (2) selecting Other -> Launch Application, (3) entering an Application Port of 8080, and (4) clicking the Your Application button.

Launch Application

You should see the following:

Local App Running

Use Ctrl+C to stop your container.

Tag the image as us.icr.io/$SN_ICR_NAMESPACE/accounts:1 and push it to the IBM Cloud registry.

Note: The environment variable SN_ICR_NAMESPACE contains your image namespace in the IBM Cloud Container Registry.

```
docker tag accounts us.icr.io/$SN_ICR_NAMESPACE/accounts:1
docker push us.icr.io/$SN_ICR_NAMESPACE/accounts:1
```

Make a Pull Request

Use git status to make sure that you have committed your changes locally in the development environment.

Use the git add command to add the new Dockerfile to the staging area.

Commit your changes using the message Added docker support.

Push your local changes to a remote branch.

Note: Use your GitHub Personal Access Token as your password in the Cloud IDE environment. You may also need to configure Git the first time you use it with:

```
git config --local user.email "you@example.com"
git config --local user.name "Your Name"
```

```
git push --set-upstream origin add-docker
```

Make a pull request, which should kick off the GitHub Actions that are now enabled on yur repository.

Once the test cases pass, merge your pull request.

Move your story to the Done column on your kanban board.

Pull the last code down to your development environment and delete your old branch.

```
git checkout main
git pull
git branch -d add-docker
```

Pick Up the Next Story

Go to your kanban board and take the next story from the top of the Sprint Backlog. It should be titled: "Deploy your Docker image to Kubernetes".

Move the story to In Progress.

Open the story and assign it to yourself.

Read the contents of the story.

Results

The story should look similar to this:

Deploy your Docker image to Kubernetes
As a service provider
I need my service to run on Kubernetes
So that I can easily scale and manage the service

Assumptions

Kubernetes manifests will be created in yaml format
These manifests could be useful to create a CD pipeline
The actual deployment will be to OpenShift

Acceptance Criteria

Given the Kubernetes manifests have been created
When I use the oc command to apply the manifests
Then the service should be deployed and run in Kubernetes

You are now ready to begin working on your second story.

Deploy to Kubernetes

Use the git checkout -b add-kubernetes command to create a new branch called add-kubernetes to work on in the development environment.

oc new-app postgresql-ephemeral

Use the oc new-app command to deploy the postgresql-ephemeral template.

Use oc get all to make sure that the postgres service is defined and the postgres pod is running.

Results

Postgres Running

You are now ready to create Kubernetes manifests for your microservice.

oc get template postgresql-ephemeral -o yaml -n openshift > postgresql.yaml

In the 'namespace' object in the 'metadata' section, please replace 'openshift' with your actual sn-labs-username which you can get using the command - oc project

3. Create the template for Postgres based on this YAML file using:

oc create -f postgresql.yaml -n sn-labs-aliimranrana

Create Manifests

Here is a tip for getting started creating manifest yaml files. You can use the kubectl or oc CLI to create a deployment or service and capture the definition in a yaml file by adding the flags --dry-run=client -o yaml. This code doesn't actually create anything (--dry-run=client) but sends output to yaml (-o yaml). Then all you need to do is redirect that to a file.

Create a manifest definiton for the account deployment using the oc create deployment command with the --dry-run -o yaml option and redirect it to a file called deploy/deployment.yaml. Specify the image that you pushed to the IBM Cloud registry and request three replicas.

```
oc create deployment accounts \
  --image=us.icr.io/$SN_ICR_NAMESPACE/accounts:1 \
  --replicas=3 \
  --dry-run=client -o yaml > deploy/deployment.yaml
```

Copied!Executed!

Your microservice needs to know the details about the postgres database that you just deployed. In particular, it needs the following environment variables: DATABASE_HOST, DATABASE_NAME, DATABASE_PASSWORD, and DATABASE_USER. Use the oc describe command to see what keys are in the secret that you can use:

```
oc describe secret postgresql
...
Data
====
database-name:      8 bytes
database-password:  16 bytes
database-user:      7 bytes
```

Edit the deploy/deployment.yaml file and use the keys that you found in the secret along with a DATABASE_HOST of postgresql to add the required environment variables to the manifest.

You can create an environment variable from a secret key like this:

```
  env:
   - name: DATABASE_NAME
       valueFrom:
       secretKeyRef:
           name: postgresql
           key: database-name
```

```
        env:
          - name: DATABASE_HOST
            value: postgresql
          - name: DATABASE_NAME
            valueFrom:
              secretKeyRef:
                name: postgresql
                key: database-name
          - name: DATABASE_PASSWORD
            valueFrom:
              secretKeyRef:
                name: postgresql
                key: database-password
          - name: DATABASE_USER
            valueFrom:
              secretKeyRef:
                name: postgresql
                key: database-user
```

Results

Apply this deployment using oc create and point it to your deploy/deployment.yaml file.

```
oc create -f deploy/deployment.yaml
```

Create a manifest definition for the account service using the oc expose command, using a type of NodePort, and a Port of 8080, and redirect it to a file called deploy/service.yaml.

```
oc expose deploy accounts \
  --type=ClusterIP \
  --port=8080 \
  --dry-run=client -o yaml > deploy/service.yaml
```

Apply this service using the oc create command and point it to your deploy/service.yaml file.

```
oc create -f deploy/service.yaml
```

Now it's time to see if everything is running. Use oc get all and filter by the level app=accounts to see your deployment running.

```
oc get all -l app=accounts
```

You should see something similar to the following:

OC get all accounts

Note: There should be a deployment, replicaset, pod, and service.

Finally, expose your service using an OpenShift route. Use the oc create command to create a route called accounts with edge termination that exposes the --service named accounts.

oc create route edge accounts --service=accounts

Use the oc get routes command to get the route that was assigned to your service.

oc get routes

Copy the URL of your route and paste it into your browser to see your application running in OpenShift.

Pick Up the Next Story

Go to your kanban board and take the next story from the top of the Sprint Backlog. It should be titled: "Create a CD pipeline to automate deployment to Kubernetes".

Move the story to In Progress.

Open the story and assign it to yourself.

Read the contents of the story.

Results

The story should look like this:

Create a CD pipeline to automate deployment to Kubernetes
As a developer
I need to create a CD pipeline to automate deployment to Kubernetes
So that developers aren't wasting their time doing it manually

Assumptions

Use Tekton to define the pipeline
It should clone, lint, test, build, and deploy the service
Deployment should be to OpenShift
It can use a manual trigger for this MVP

Acceptance Criteria

Given the CD pipeline has been created
When I trigger the pipeline run
Then I should see the accounts service deployed to OpenShift

You are now ready to begin working on your story.

Overview and Setup

In reading your story, you see that the assumptions state the following:

Assumptions

Use Tekton to define the pipeline
It should clone, lint, test, build, and deploy the service
Deployment should be to OpenShift
It can use a manual trigger for this MVP
It looks like you will need to create a pipeline task for each of these:

Task Name	Build After	Message
clone	-	Cloning the repo $(params.repo-url)
lint	clone	Calling Flake8 linter
tests	clone	Running unit tests with PyUnit
build	lint, tests	Building image for $(params.repo-url)
deploy	build	Deploying $(params.branch) branch of $(params.repo-url)

Luckily, the developer that worked on this before you started to create a pipeline but didn't get very far. They did, however, create a starter Pipeline in pipeline.yaml with a clone task, and defined a PersistentVolumeClaim in the pvc.yaml file to define a workspace.

Let's see if what we have already works.

Your Task

Change into your project directory: cd devops-capstone-project.

Create a new branch called cd-pipeline to work on in the development environment.

Run nosetests and make sure that all of the test cases are passing. Fix any that fail before proceeding.

Use the oc create command on the tekton/pvc.yaml file to create a workspace for your pipeline to use.

oc create -f tekton/pvc.yaml

Use the oc apply command on the tekton/tasks.yaml file to create the tasks for your pipeline to use.

oc apply -f tekton/tasks.yaml

Use the oc apply command on the tekton/pipeline.yaml file to create the initial pipeline.

oc apply -f tekton/pipeline.yaml

If you edit the pipeline.yaml file, you will see that the clone task requires the git-clone task from Tekton Hub. Use the tkn hub install task command to install that now.

tkn hub install task git-clone

Note: If the above command returns a error due to Tekton Version mismatch, please run the below command to fix this.

kubectl apply -f https://raw.githubusercontent.com/tektoncd/catalog/main/task/git-clone/0.9/git-clone.yaml

Use the tkn pipeline start command to start the pipeline and make sure the initial pipeline definitions work.

```
tkn pipeline start cd-pipeline \
   -p repo-url="https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git" \
   -p branch="main" \
   -w name=pipeline-workspace,claimName=pipelinerun-pvc \
   -s pipeline \
   --showlog
```

Results

You should see the following output that proves that the initial pipeline is working. Now it is safe to start adding more tasks to it.

Initial Pipeline Run

Just to be sure it ran successfully, use tkn pipelinerun ls and check the status column, which should show Succeeded.

tkn pipelinerun ls

It should return:

Initial Pipeline Status

Create the Lint Task

The first task that you need to add is the lint task. You are going to use flake8 to lint your code. Luckily, Tekton Hub has a flake8 task that you can install and use:

You can use the following button to open the pipeline.yaml file to start editing it:

Open pipeline.yaml in IDE

Your Task

Use the tkn hub install command to install the task named flake8 into your Kubernetes namespace.

You should see output similar to this:

Install flake8

tkn hub install task flake8

Now you will modify the pipeline.yaml file to use the new flake8 task. Start by copying and pasting the existing clone task and change the name to lint.

In reading the documentation for the flake8 task, you notice that it requires a workspace named source. You should change the workspace name to match. You also need to change the taskRef to flake8.

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
```

You now need to set up the parameters. The flake8 task takes an optional image parameter that you will use, and the args parameter for flake8 to run. Change the existing parameters to use python:3.9-slim as the image, and pass the following args parameters to flake8: --count --max-complexity=10 --max-line-length=127 --statistics.

Note: Replace "..." with each parameter you want to pass.

```
  - name: {name here}
```

```
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
    params:
    - name: image
      value: {image name here}
    - name: args
      value: ["...","...","...","..."]
```

Finally, you need to define when it should run. Add the runAfter: tag and indicate that it should run after the previous clone task.

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
    params:
    - name: image
      value: {image name here}
    - name: args
      value: {arguments here}
    runAfter:
      - {task before}

  - name: lint
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: flake8
    params:
    - name: image
      value: "python:3.9-slim"
    - name: args
      value: ["--count","--max-complexity=10","--max-line-length=127","--statistics"]
    runAfter:
      - clone
```

Start the Pipeline

Now it's time run the pipeline and make sure that the lint task works.

Apply the changes:

```
oc apply -f tekton/pipeline.yaml
```

Start the pipeline:

```
tkn pipeline start cd-pipeline \
      -p repo-url="https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git" \
   -p branch="main" \
      -w name=pipeline-workspace,claimName=pipelinerun-pvc \
   -s pipeline \
   --showlog
```

Check the Run Status
It may take a long time for the pipeline to run. The logs should display on the screen but if they don't, you can see the pipeline run status by opening a new terminal and listing the pipeline runs with:

```
tkn pipelinerun ls
```

You can check the logs of the last run with:

```
tkn pipelinerun logs --last
```

Commit Your Changes
Becasue the Cloud IDE is ephemeral, it may timeout and you will lose all of your work. It is strongly recommended that you commit your changes and push them to your remote branch so that you can pull them from GitHub if needed.

```
git commit -am 'added link task'
git push --set-upstream origin cd-pipeline
```

Now you are ready to move on to the next exercise.

Create a Test Task

The next stage of the pipeline you need to create is a test task. In your development environment, you use Nose to run your tests using the nosetests command. Unfortunately, there are no tasks in the Tekton Hub for nosetests, so you will write your own.

In this exercise, you will update the tasks.yaml file adding a new task called nose that uses the shared workspace for the pipeline and runs nosetests in a python:3.9-slim image as a shell script as seen in the CI/CD course video.

Here is a bash script to install the Python requirements and run the nosetests. You can use this as the shell script in your new task:

#!/bin/bash
set -e
python -m pip install --upgrade pip wheel
pip install -qr requirements.txt
nosetests -v --with-spec --spec-color
Copied!
You can use the following button to open the tasks.yaml file to start editing it:

Open tasks.yaml in IDE

Your Task:
First, create a new task in the tasks.yaml file. Start by copying and pasting the existing echo task and change the name to nose.

Remember, each new task must be separated using three dashes — on a separate line.

Replace the `{here}` placeholders with the real values:
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: {task name here}
spec:

Next, it's a good idea to add a description to yor task so that others who may use it know what it does. Change the description: tag to be: This task will run nosetests on the provided input.

Replace the `{here}` placeholders with the real values:

```
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: {task name here}
spec:
  description: {description here}
```

Next, you need to include the workspace that has the code that you want to test. Since flake8 uses the name source, you can use that for consistency. Add a workspace named source.

Replace the `{here}` placeholders with the real values:

```
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: {task name here}
spec:
  description: {description here}
  workspaces:
    - name: {workspace name here}
```

It might be a good idea to allow the passing in of different arguments to nosetests, so create a parameter called args just like the flake8 task has, and give it a description:, make the type: a string, and a default: with the verbose flag -v as the default.

Replace the `{here}` placeholders with the real values:

```
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: {task name here}
spec:
  description: {description here}
  workspaces:
    - name: {workspace name here}
  params:
    - name: {argument name here}
      description: Arguments to pass to nose
      type: {type here}
      default: {default args here}
```

Quite often, tests require a test database. It might be a good idea to allow the passing in of a database url. Add a new parameter called database_uri just like the args parameter. Give it a description:, make the type: a string, and a default: with SQLite ("sqlite:///test.db") in case they don't have a separate database server.

Replace the `{here}` placeholders with the real values:

```
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: {task name here}
spec:
  description: {description here}
  workspaces:
    - name: {workspace name here}
  params:
    - name: {argument name here}
      description: Arguments to pass to nose
      type: {type here}
      default: {default args here}
    - name: {database uri name here}
      description: Database connection string
      type: {type here}
      default: {default database uri here}
```

Finally, you need to specify the steps:, and there is only one. Give it the name nosetests.

Replace the `{here}` placeholders with the real values:

```
spec:
  ...
  steps:
    - name: {place name here}
```

Have it run in a python:3.9-slim image.

Replace the `{here}` placeholders with the real values:

```
spec:
  ...
  steps:
```

```
  - name: {place name here}
    image: {image name here}
```

Also, specify workingDir as the path to the workspace you defined (i.e., $(workspaces.source.path)).

Replace the `{here}` placeholders with the real values:

```
spec:
  ...
  steps:
    - name: {place name here}
      image: {image name here}
      workingDir: {working path here}
```

Next, add an environment variable called DATABASE_URI that gets its value from the new parameter you just created: $(params.database_uri).

Replace the `{here}` placeholders with the real values:

```
spec:
  ...
  steps:
    - name: {place name here}
      image: {image name here}
      workingDir: {working path here}
      env:
        - name: {environment variable name here}
          value: {value from parameters here}
```

Then, paste the script from above in the script: parameter and substitute $(params.args) for the parameters passed into nosetests instead of the hard-coded ones.

Replace the `{here}` placeholders with the real values:

```
spec:
  ...
  steps:
    - name: {place name here}
      image: {image name here}
      workingDir: {working path here}
      env:
        - name: {environment variable name here}
```

```
        value: {value from parameters here}
    script: |
      {paste the script from above here}
```

Your new task should look similar to this:

```
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: nose
spec:
  description: This task will run nosetests on the provided input.
  workspaces:
    - name: source
  params:
    - name: args
      description: Arguments to pass to nose
      type: string
      default: "-v"
    - name: database_uri
      description: Database connection string
      type: string
      default: "sqlite:///test.db"
  steps:
    - name: nosetests
      image: python:3.9-slim
      workingDir: $(workspaces.source.path)
      env:
        - name: DATABASE_URI
          value: $(params.database_uri)
      script: |
        #!/bin/bash
        set -e
        echo "***** Installing dependencies *****"
        python -m pip install --upgrade pip wheel
        pip install -qr requirements.txt
        echo "***** Running nosetests with: $(params.args)"
        nosetests $(params.args)
```

Apply Your Changes
Apply these changes to your cluster:

oc apply -f tekton/tasks.yaml

You should see the following output:

Apply Tasks

Commit Your Changes

Becasue the Cloud IDE is ephemeral, it may timeout and you will lose all of your work. It is strongly recommended that you commit your changes and push them to your remote branch so that you can pull them from GitHub if needed.

git commit -am 'added nose task'
git push

Now you are ready to move on to the next exercise.

Add the Test Task to the Pipeline

Now it's time to update the pipeline to add the test task that uses the new nose task that you have created and then run the pipeline to see it work.

You can use the following button to open the pipeline.yaml file to start editing it:

Open pipeline.yaml in IDE

Your Task

Start by editing the pipeline.yaml file to add the new test task. Start by copying and pasting the lint task and change the name to tests.

```
  - name: {name here}
```

Leave the workspace named source and change the taskRef: from lint to reference your new nose task.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
```

Change the image parameter to database_uri with a value of sqlite:///test.db.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
    params:
    - name: {database uri parameter here}
      value: {value of database uri here}
```

Change the value of the existing args parameter to specify the arguments to pass to the tests as "-v --with-spec --spec-color".

```
  - name: {name here}
    workspaces:
```

```
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
    params:
    - name: {database uri parameter here}
      value: {value of database uri here}
    - name: args
      value: {arguments enclosed in double quotes here}
```

Finally, specify that this task should runAfter: the clone step.

Notice that you are running lint and tests in parallel because they do not depend on each other. This will make the pipeline run faster.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
    params:
    - name: {database uri parameter here}
      value: {value of database uri here}
    - name: args
      value: "{arguments enclosed in double quotes here}"
    runAfter:
      - {task before}
```

Your new step should look similar to this:

```
  - name: tests
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: nose
    params:
    - name: database_uri
      value: "sqlite:///test.db"
    - name: args
      value: "-v --with-spec --spec-color"
```

```
    runAfter:
      - clone
```

Commit Your Changes
Because the Cloud IDE is ephemeral, it may timeout and you will lose all of your work. It is strongly recommended that you commit your changes and push them to your remote branch so that you can pull them from GitHub if needed.

```
git commit -am 'added test pipeline task'
git push
```

Now you are ready to move on to the next exercise.

Start the Pipeline

Now it's time run the pipeline and make sure that the tests task works. The tests task will need a PostgreSQL database to run against. If it isn't running in your OpenShift cluster from the last lab, you must provision it again.

Your Task

Use the oc get svc postgresql command to see if the PostgreSQL service is defined.

oc get svc postgresql

If it is not found, use the oc new-app command to deploy the postgresql-ephemeral template.

oc new-app postgresql-ephemeral

Use oc get pods to make sure that the postgres service is defined and the postgres pod is running.

oc get pods

You should see the following:

Postgres Running

You are now ready to apply the changes to your pipeline.

oc apply -f tekton/pipeline.yaml

You should see the message: pipeline.tekton.dev/cd-pipeline configured

Start the Pipeline

Make sure that the GITHUB_ACCOUNT environment variable is set to the name of your GitHub account, and use the tkn pipeline start command to run the pipeline.

```
tkn pipeline start cd-pipeline \
      -p repo-url="https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git" \
   -p branch="main" \
      -w name=pipeline-workspace,claimName=pipelinerun-pvc \
   -s pipeline \
   --showlog
```

Note that because you specified that lint and tests can run in parallel, you will see the log output of those two tasks intermixed.

Create a Build Task

Before you can deploy your application, you need to build a Docker image and push it to an image registry. You search Tekton Hub for the word "build" and you see there is a task called buildah that will build images so you decide to use it in your pipeline to build your code.

Instead of installing it yourself, you first check the ClusterTasks in your cluster to see if it already exists. Luckily, the OpenShift environment you are using already has buildah installed as a ClusterTask. A ClusterTask is installed cluster-wide by an administrator, and anyone can use it in their pipelines without having to install it themselves.

You can always check what ClusterTasks are installed with this command:

tkn clustertask ls

Now, you will update the pipeline.yaml file to use the new buildah task. You can use the following button to open the pipeline.yaml file to start editing it:

Open pipeline.yaml in IDE

Your Task
Start by editing the pipeline.yaml file to add the new build task. Start by copying and pasting the tests task and changing the name to build.

```
  - name: {name here}
```

The workspace name source from the tests task is fine, but the taskRef needs to be changed to buildah, and you must add an additional line below it to indicate that it has a kind: of ClusterTask.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: {kind here}
```

Delete the database_uri parameter. It is not needed for this task.

Next, change the args parameter to IMAGE and set the value to "$(params.build-image)" to pick up the build-image name from the pipeline parameters.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: {kind here}
    params:
    - name: {parameter name here}
      value: (params value here}
```

Now, you need to add a parameter named build-image to the existing list of parameters at the top of the pipeline under spec.params.

```
spec:
  params:
    - name: repo-url
    - name: branch
      default: main
    - name: {new parameter name here}
```

Finally, you must specify what task or tasks this should run after. Because both tests and lint ran in parallel, you want to wait until both complete before bulding the image. Change the taskRef to specify both tests and lint.

```
  - name: {name here}
    workspaces:
      - name: source
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: {kind here}
    params:
    - name: {parameter name here}
      value: (params value here}
    runAfter:
      - {first task here}
      - {second task here}
```

Check that your step looks like this:

```
spec:
```

```
  params:
    ... other parameters ...
    - name: build-image

    - name: build
      workspaces:
        - name: source
          workspace: pipeline-workspace
      taskRef:
        name: buildah
        kind: ClusterTask
      params:
      - name: IMAGE
        value: "$(params.build-image)"
      runAfter:
        - tests
        - lint
```

Apply the Changes
You are now ready to apply the changes to your pipeline.

```
oc apply -f tekton/pipeline.yaml
```

You should see the message: pipeline.tekton.dev/cd-pipeline configured

Commit Your Changes
Because the Cloud IDE is ephemeral, it may timeout and you will lose all of your work. It is strongly recommended that you commit your changes and push them to your remote branch so that you can pull them from GitHub if needed.

```
git commit -am 'added build task'
git push
```

Start the Pipeline
When you start the pipeline, you need to pass in the build-image parameter, which is the name of the image to build.

This will be different for every learner that uses this lab. Here is the format:

```
image-registry.openshift-image-registry.svc:5000/$SN_ICR_NAMESPACE/accounts:latest
```

Notice the variable $SN_ICR_NAMESPACE in the image name. This is automatically set to point to your container namespace. There is nothing for you to do here.

Make sure that the GITHUB_ACCOUNT environment variable is set to the name of your GitHub account, and use the tkn pipeline start command to run the pipeline.

```
tkn pipeline start cd-pipeline \
    -p repo-url="https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git" \
    -p branch=main \
    -p
build-image=image-registry.openshift-image-registry.svc:5000/$SN_ICR_NAMESPACE/account
s:1 \
    -w name=pipeline-workspace,claimName=pipelinerun-pvc \
    -s pipeline \
    --showlog
```

Check that everything ran to completion (this may take a while).

Add the Deploy Task to the Pipeline

You are now at the deploy step, which is the last step in your CD pipeline. For this step, you will use the OpenShift client to deploy your Docker image to an OpenShift cluster.

Knowing that you want to deploy to OpenShift, you search Tekton Hub for "openshift" and you see there is a task called openshift-client that will execute OpenShift commands on your cluster. You decide to use the openshift-client task in your pipeline to deploy your image.

First, you need to update the pipeline.yaml file to reference the new openshift-client ClusterTask that you want to use in the deploy pipeline. Then you will need to update your deploy/deployment.yaml file to substitute the name of the image to deploy.

In the previous steps, you simply changed the name of the reference to the task, but since the openshift-client task is installed as a ClusterTask, you need to add the statement kind: ClusterTask under the name so that Tekton knows to look for a ClusterTask and not a regular Task.

Your Task

Start by editing the pipeline.yaml file to add the new deploy task. Start by copying and pasting the build task and changing the name to deploy.

```
  - name: {name here}
```

Copied!
Next, change the name: of the workspaces: to manifest-dir as specified in the documentation of the openshift-client on Tekton Hub.

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
```

Then, change the taskRef from buildah to openshift-client and leave the kind: as ClusterTask to indicate that this is a ClusterTask.

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: ClusterTask
```

Now, change the IMAGE parameter to the name SCRIPT and specify the oc command to apply your Kubernetes manifests from the previous lab. Before you do, you will need to modify your deployment.yaml file to have an image name placeholder and use sed to change it to the image name for the pipeline before execution.

Here is the script to use (you will modify the deployment.yaml file in a later step):

```
echo "Updating manifest..."
sed -i "s|IMAGE_NAME_HERE|$(params.build-image)|g" deploy/deployment.yaml
cat deploy/deployment.yaml
echo "Deploying to OpenShift..."
oc apply -f deploy/
oc get pods -l app=accounts
```

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: ClusterTask
    params:
    - name: {parameter name here}
      value: |
        {script commands here}
```

Finally, specify that this task should runAfter: the build task.

```
  - name: {name here}
    workspaces:
      - name: {workspace name here}
        workspace: pipeline-workspace
    taskRef:
      name: {task name here}
      kind: ClusterTask
    params:
    - name: {parameter name here}
      value: |
        {script commands here}
    runAfter:
      - {task to run after here}
```

If you changed everything correctly, the full deploy task in the pipeline should look like this:

```
  - name: deploy
    workspaces:
      - name: manifest-dir
        workspace: pipeline-workspace
    taskRef:
      name: openshift-client
      kind: ClusterTask
    params:
    - name: SCRIPT
      value: |
        echo "Updating manifest..."
        sed -i "s|IMAGE_NAME_HERE|$(params.build-image)|g" deploy/deployment.yaml
        cat deploy/deployment.yaml
        echo "Deploying to OpenShift..."
        oc apply -f deploy/
        oc get pods -l app=accounts
    runAfter:
      - build
```

Apply the Pipeline
Apply the same changes you just made to pipeline.yaml to your cluster:

kubectl apply -f tekton/pipeline.yaml

If it applied correctly, you can commit your changes. That doesn't mean it will run without errors, but it's a good idea to commit what you have before you start debugging.

Update Your Deployment Manifest

There is one last thing that you need to do before you can start the pipeline. You must edit your deploy/deployment.yaml file to specify a placeholder for the name of the image that you just built in the build step.

Open deployment.yaml in IDE

Find the image: tag and change its value to IMAGE_NAME_HERE.

For example:

```
spec:
  template:
    spec:
```

```
    containers:
    - image: IMAGE_NAME_HERE
      name: accounts
```

Commit Your Changes

Because the pipeline is going to clone your Git repository and you changed deploy/deployment.yaml, you must push all of your code changes back to your remote branch in GitHub. You will also need to use this branch name when you run the pipeline. If you followed the instructions, the current branch should be: cd-pipeline.

```
git commit -am 'added deploy task'
git push
```

Apply Changes and Run the Pipeline

Now that you have built out the entire pipeline, it is time to run it. If you were in a persistent environment you would set up a webhook in GitHub so that whenever your code changes, the CD pipeline would run after all the tests and checks had passed.

Unfortunately, the lab environnement is ephemeral and the best we can do is start the pipeline manually.

Start the Pipeline

When you start the pipeline, you will need to use the name of the branch that you are working on, which should be cd-pipeline.

Now, start the pipeline to see your new deploy task run. Use the Tekton CLI pipeline start command to run the pipeline, passing in the parameters repo-url, branch, and build-image using the -p option. Specify the workspace pipeline-workspace and persistent volume claim pipelinerun-pvc using the `-w option:

Your Task

First, make sure that you have set the environment variable GITHUB_ACCOUNT to contain the name of your GitHub account.

Use the tkn pipeline start command to start the pipeline and see if your deploy task works.

```
tkn pipeline start cd-pipeline \
    -p repo-url="https://github.com/$GITHUB_ACCOUNT/devops-capstone-project.git" \
    -p branch=cd-pipeline \
    -p
build-image=image-registry.openshift-image-registry.svc:5000/$SN_ICR_NAMESPACE/account
s:1 \
    -w name=pipeline-workspace,claimName=pipelinerun-pvc \
    -s pipeline \
    --showlog
```

Check the Deployment

Now, check to see if the deployment is running. Use the oc command to check that your deployment is in a running state.

```
oc get all -l app=accounts
```

Evidence
For the evidence, you will capture the logs of the piepline run in a file called pipelinerun.txt

Open your terminal and issue the command tkn pipelinerun logs -L and redirect it to a file called: pipelinerun.txt.

tkn pipelinerun logs -L > pipelinerun.txt

Open this file in the Cloud IDE editor and copy and paste it into a file of the same name on your computer so that you can upload it as evidence.